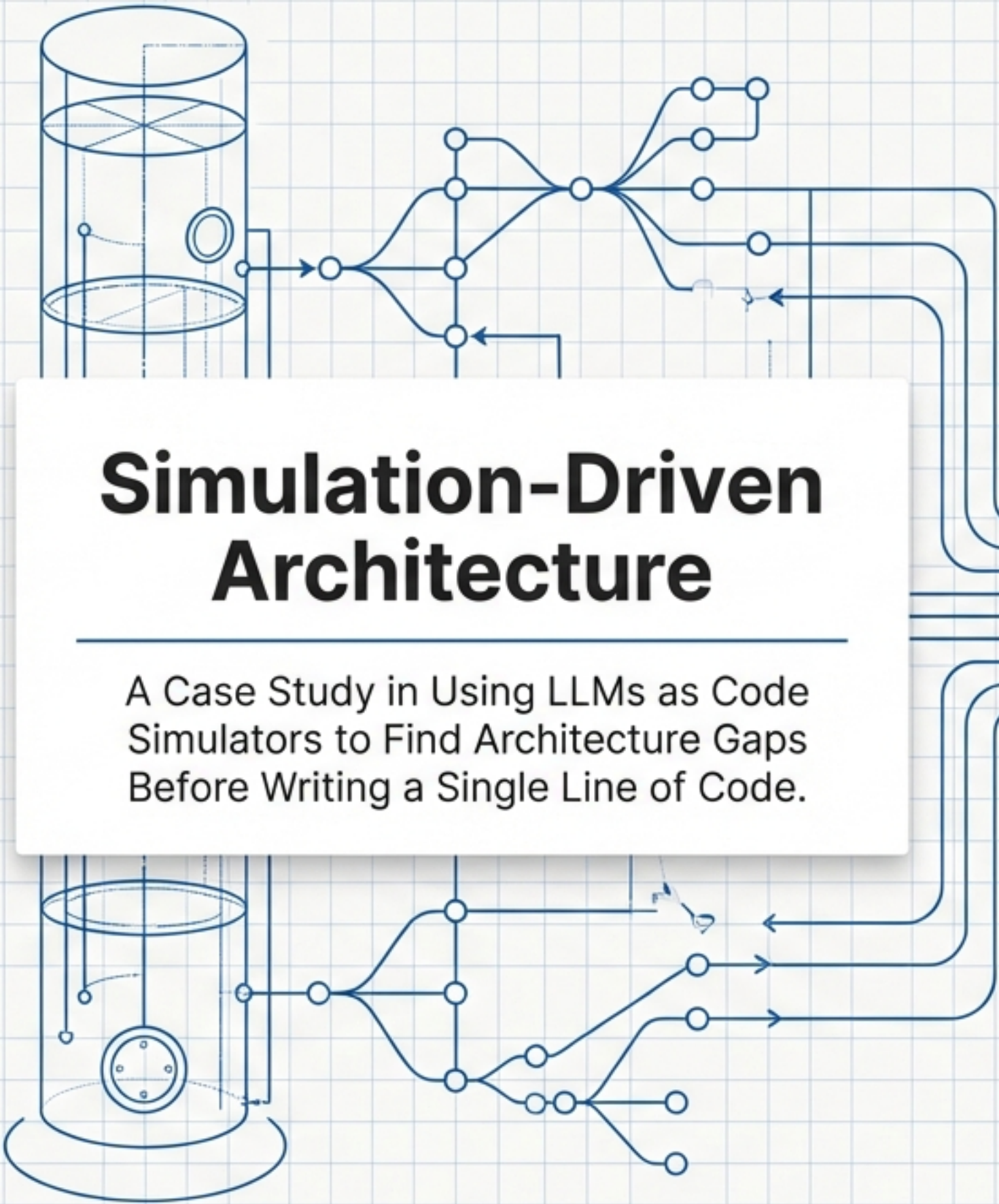




Simulation-Driven Architecture

A Case Study in Using LLMs as Code Simulators to Find Architecture Gaps Before Writing a Single Line of Code.

> INITIATING SIMULATION SEQUENCE [v2.4.1]

|--> DATA_FLOW: `

| [SOURCE: ARCH_MODEL,
| TARGET: LLM_SIM]

|--> VALIDATION: [SUCCESS]

| |--> CHECK: GAP_ANALYSIS [OK]

| |--> CHECK: LOAD_TEST [OK]

|--> EXECUTION: `

| STATUS: RUNNING,

| MEMORY_USAGE: 450MB,

| LATENCY: 12ms]

|--> WARNING: POTENTIAL BOTTLENECK DETECTED AT NODE_42

{
- DATA {

|--> DATA_FLOW: ` [SOURCE: ARCH_MODEL,
| TARGET: LLM_SIM

}

|--> VALIDATION: GAP_ANALYSIS

}

})

-- API CALLS

|--> DATA_FLOW: [SOURCE: ARCH_MODEL,
| TARGET: LLM_SIM

|

|--> EXECUTION: EXECU.TEST

!--> DATA: [
{

STATUS: RUNNING,
MEMORY_USAGE: 450MB,
LATENCY: 12ms

}

]

}

IF CONDITIONAL > {
ERRORS: {

STATUS: RUNNING,
MEMORY_USAGE: 450MB,
LATENCY: 12ms

}

}

}

The Architecture Was Ready. Or So We Thought.

The Project

System

SGraph Send CLI (Zero-knowledge encrypted file vault)

Scope Expansion

Adding branches, multi-user collaboration, merge, and conflict resolution

The Status

3 architecture briefs written by project lead. ✓

2 in-depth exploration docs drafted. ✓

8-9 week timeline approved. ✓

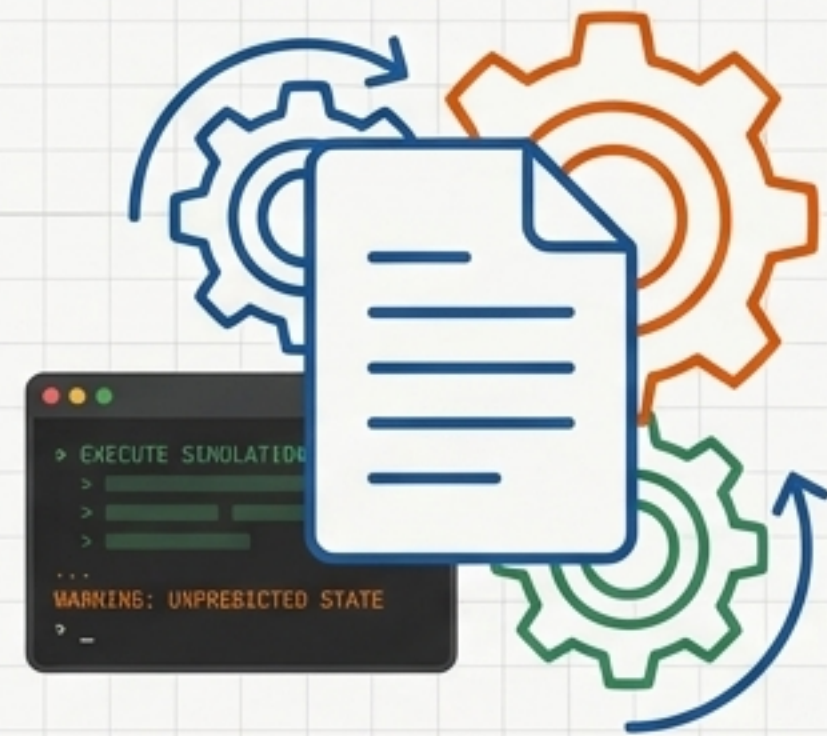
GO

The team was ready to code. Then, we decided to simulate it first.

“An architecture you can’t simulate is an architecture you don’t fully understand.”



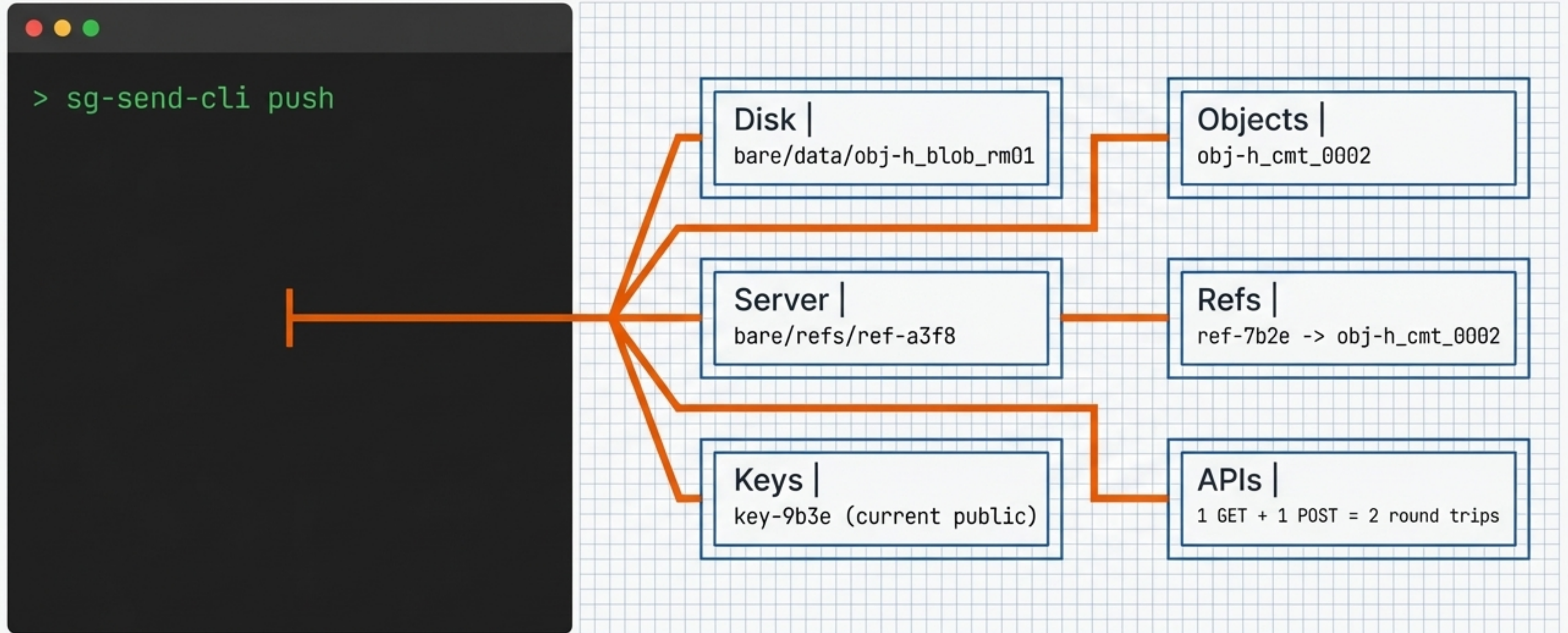
Static Analysis



Paper Execution

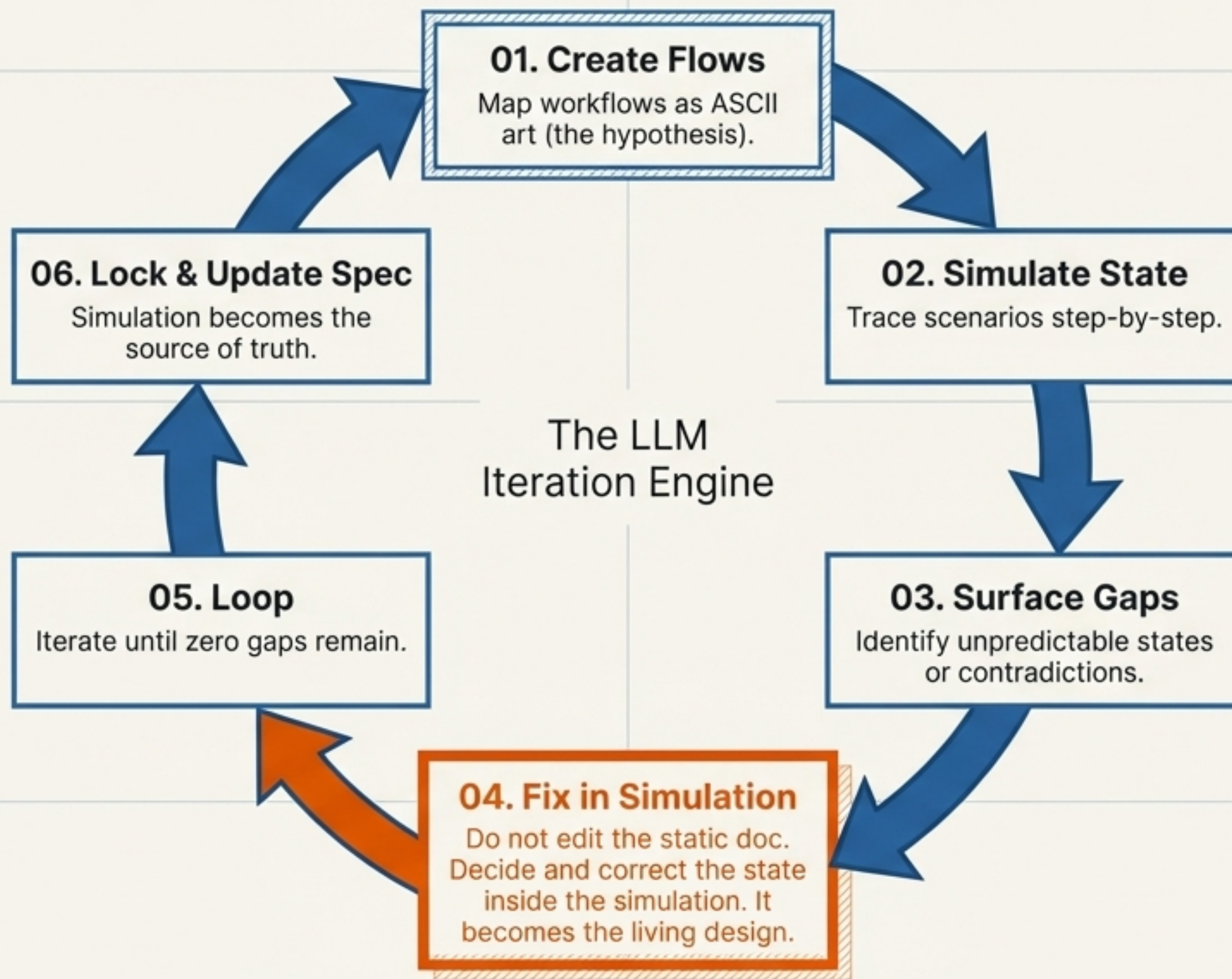
If you cannot predict exactly what happens at every step—every file, key, and API call—unstated assumptions remain.

Executing Architecture on Paper



Claude acts as the CLI engine. We issue commands; the LLM predicts the exact state.

The 6-Step Simulation Blueprint



Stress-Testing the Unwritten Code

The simulation was tested against concrete scenarios, not theoretical ideals. We tracked approximately 20 distinct steps across these five workflows.

01

Solo Workflow

[init, add files, commit, push]

02

Two-User Collaboration

[clone, diverge, push, merge]

03

Conflict

[both edit the exact same file]

04

Change Pack

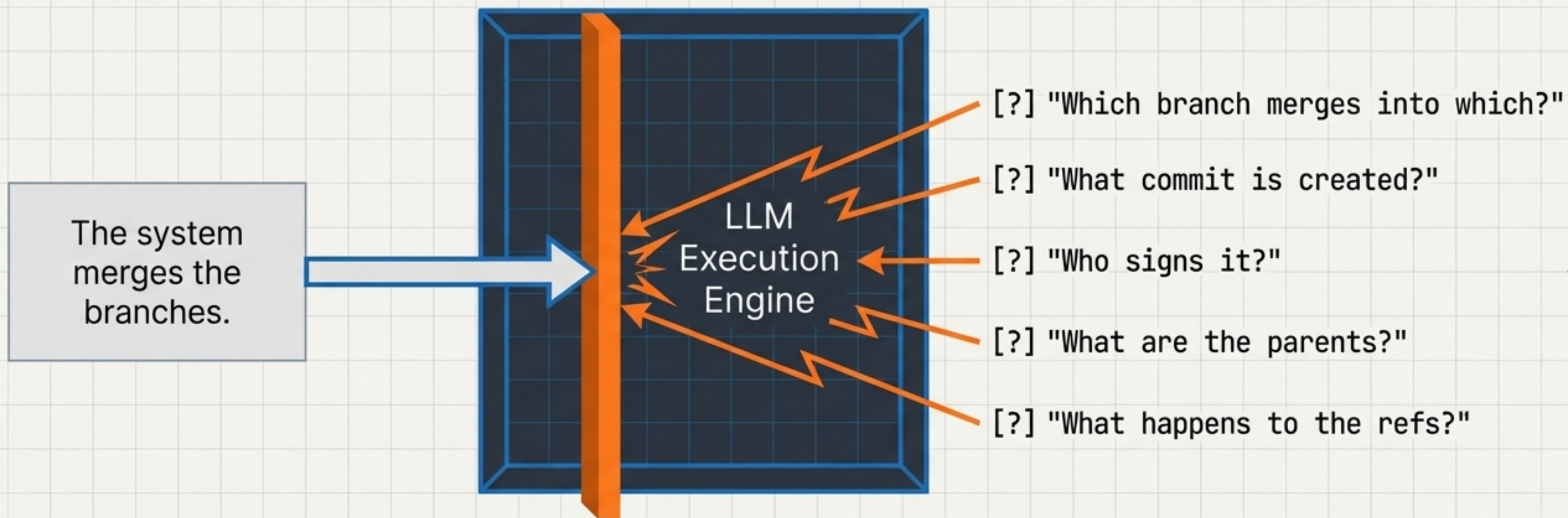
[keyless creator, zero vault access]

05

Status Detection

[local vs remote state sync]

Vagueness = Runtime Error



Key Takeaway: An architecture doc can comfortably hold vague statements. An LLM simulator halts. It cannot proceed without a concrete decision, forcing the human to tighten the spec immediately.

The ROI of a Single Session

14

Architectural Gaps Found

6

Revisions (v1 through v6)

5

Major Design Improvements

0

Lines of Wasted Code

1

Claude Session (~1000 lines final spec)

Estimated Implementation Bugs Prevented: 6-10 (based on gap severity)

Diagnosing the 14 Gaps

Static Gaps

(Found in doc review)

Total: 3 Gaps

- Clone checks out an empty main branch
- Change pack auth model blocks keyless creators
- Missing trigger for merging into main

Early developer confusion; workflow blockers.

Execution Gaps

(Found ONLY in simulation)

Total: 11 Gaps

- Merge-then-fetch ordering creates discarded commits
- Init commit signing chain starts with the wrong key
- Batch endpoint spec requires 6+ round trips instead of 2

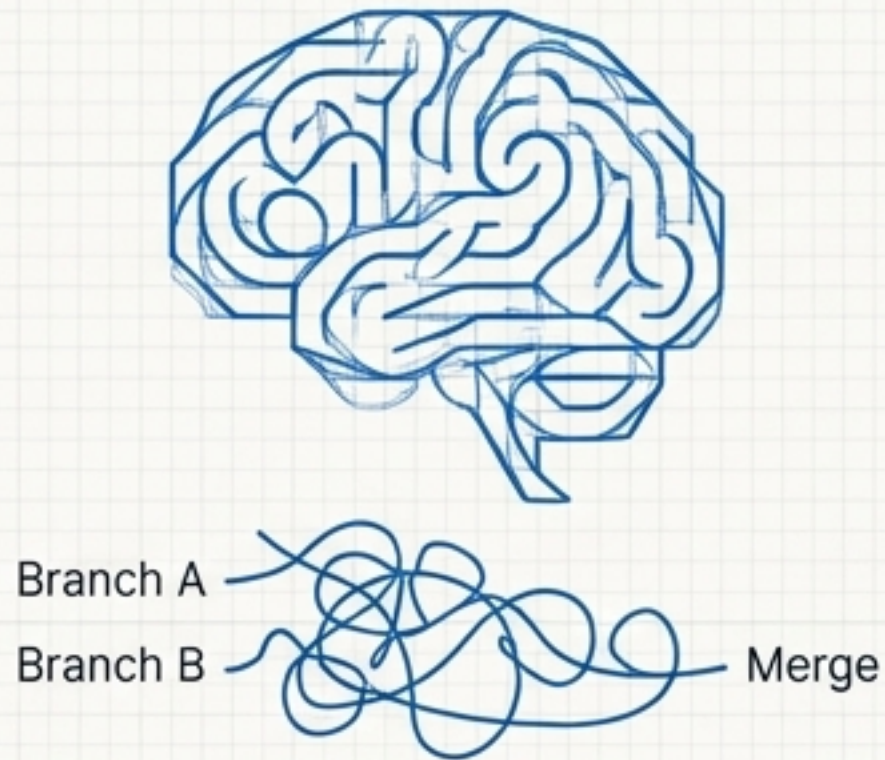
Deep, systemic, hard-to-diagnose runtime bugs that erode system trust.

Architecture Evolution: Before & After Simulation

Architectural Feature	Static Design (Before)	Simulated Reality (After)	Why it Matters
Conflict Location	Conflicts happen on named branch.	Conflicts moved to clone branch.	Prevents global blocking; enables collaborative workflows.
Vault Structure	Objects inside <code>.sg_vault/objects/</code>	Portable bare/ directory.	Makes portable-vault concept explicit.
Branch Naming	Cleartext (refs/heads/current)	Zero-knowledge opaque IDs (ref-a3f8)	Prevents server from inferring branch names.
Timestamp Format	ISO datetime strings.	Timestamp_Now (Uint milliseconds).	Compact, sortable, strips human-readable temporal metadata.

The Perfect Division of Labour

The Human Steers

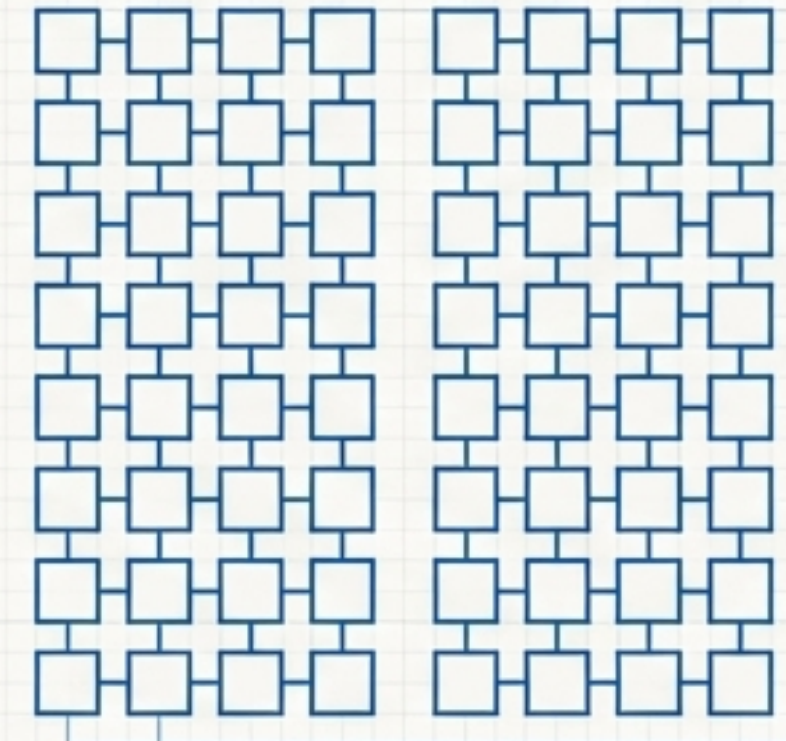


Role: Provides design judgement and architectural philosophy.

Example: Conflicts should live on the clone branch, not the named branch.

Limitation: Cannot hold massive state context in short-term memory.

The LLM Executes

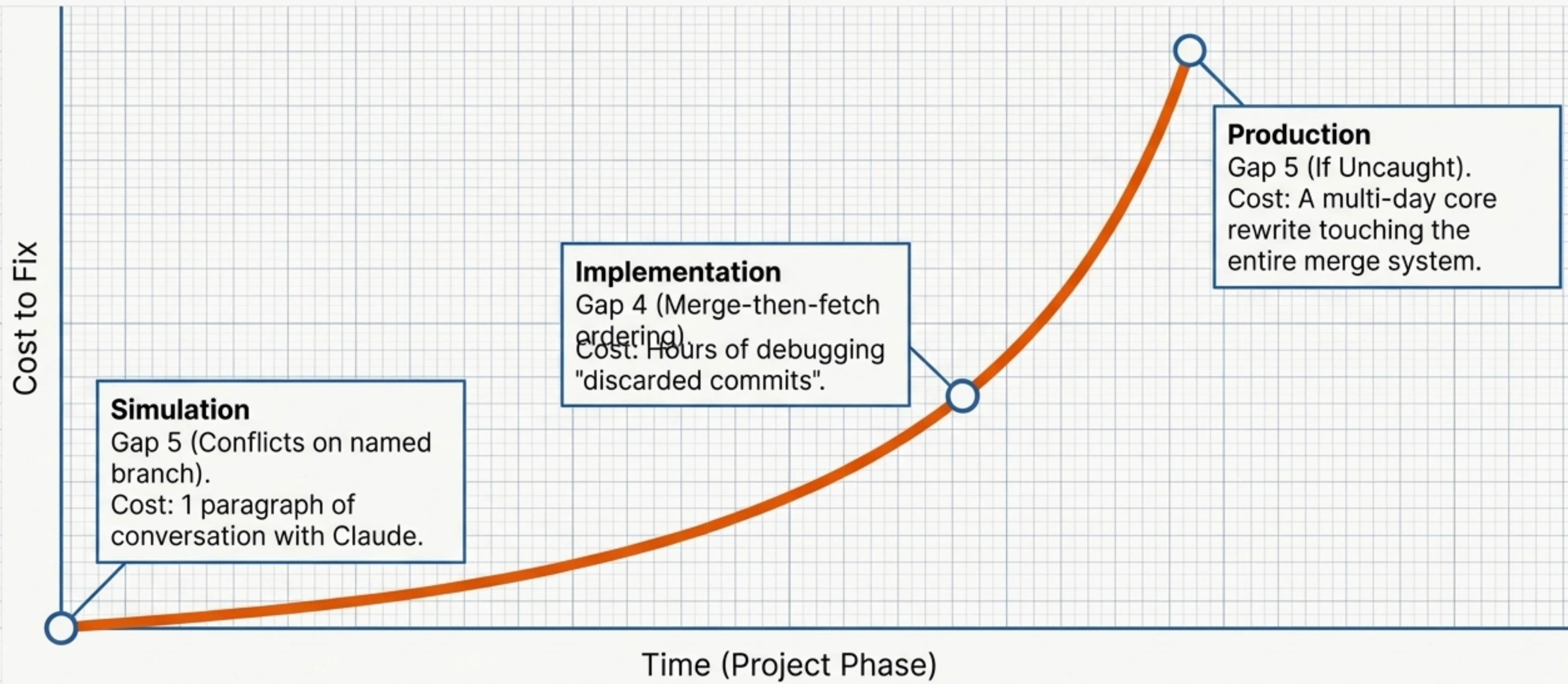


Role: Provides exhaustive state tracking and literal execution.

Example: Given that decision, here are the exact 28 files that exist on the server.

Limitation: Cannot make philosophical design choices.

The Compounding Cost of Late Discovery



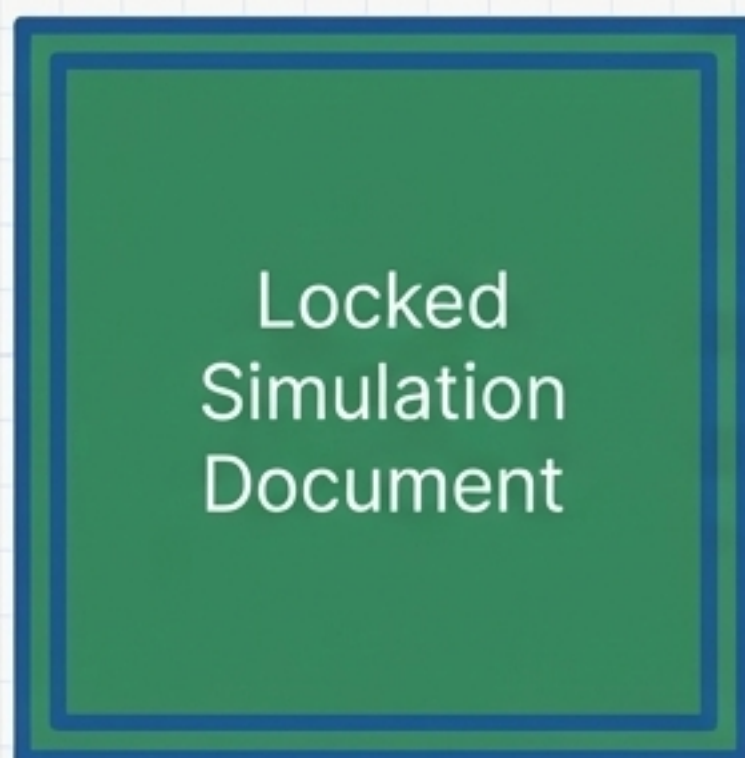
Catching a design-level refactor in simulation saves weeks of engineering time.

The Paradigm Shift: Traditional vs. Simulation-Driven

Architecture Explainer Docs	Source of Truth	The Locked Simulation State
Static Peer Review	Validation Method	Active Execution against Scenarios
Human Mental Model	State Tracking	Exhaustive LLM Register
Found during integration coding	Gap Discovery	Found before the first line is written
Vague text descriptions	Final Output	Concrete, testable QA predictions

Formulate Hypothesis → Execute via LLM → Resolve in Loop → Lock Spec

Agentic Handoff & The QA Oracle



Development & Operations

Dev creates implementation plans directly from simulation steps. DevOps designs exact batch endpoints mapped in the spec.



Application Security (AppSec)

Validates zero-knowledge properties and cryptographic key lifecycles natively embedded in the simulation.



The QA Oracle

QA extracts the predictions table. After coding, the actual CLI is run against these exact scenarios. Deviations mean a bug, or an unstated assumption.

The Most Expensive Line of Code

“The most expensive line of code is the one you write based on an architecture with an unstated assumption. The fix isn’t changing one line—it’s rethinking the foundation.”

14 Gaps Found.
0 Lines of Code Written.
1 Session.